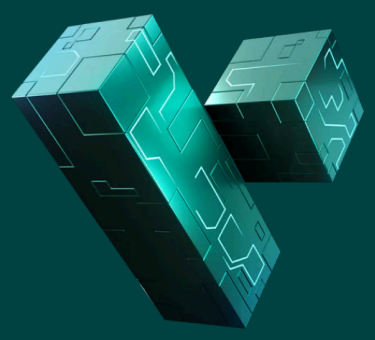
VANAR

BlOCKCHAIN PROTOCOL SECURITY

Audit Report



IMMUNEBYTES

12 February 2024

Introduction **3**
About Vanarchain 3
About ImmuneBytes 3
Documentation Details **4**
Audit Goals **4**
Audit Process & Methodology **5**
Project Summary **7**
Overview 7
Technical Features 7
Conclusion 7
Audit Details **8**
Security Level References **9**
Severity Status References **10**
Severity Status **10**
Finding **11**
Critical Severity Issues **12**
High Severity Issues **12**
Medium severity issues **15**
Low severity issues **20**
Informational **22**
Concluding Remarks **25**
Disclaimer **25**

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Introduction

About Vanarchain

Blockchain technology has experienced significant growth over the past decade, offering transformative potential across various sectors. However, challenges such as high transaction costs, sluggish speeds, and complex onboarding processes have hindered its widespread adoption, particularly in banking, finance, gaming, entertainment, real estate, healthcare, and microtransactions. In response, Vanar emerges as a groundbreaking solution, aiming to mainstream web3 by prioritizing speed, affordability, and user accessibility. With a focus on ultra-fast transactions, astonishingly low costs, and simplified onboarding, Vanar addresses the pain points of industries like entertainment, gaming, VR, AR, AI, and the metaverse. Positioned as a leader in blockchain innovation, Vanar seeks to redefine the technology for a new era, driving mass adoption and accessibility.

Visit https://vanarchain.com/ to know more about it.

About ImmuneBytes

ImmuneBytes is a security start-up that provides professional services in the blockchain space. The team has hands-on experience conducting security audits, penetration testing, and security consulting. ImmuneBytes's security auditors have worked on various A-league projects and understand DeFi projects like AAVE, Compound, 0x Protocol, Uniswap, and dydx.

The team has secured 205+ blockchain projects by providing security services on different frameworks. The ImmuneBytes team helps start-ups with detailed system analysis, ensuring security and managing the overall project.

Visit http://immunebytes.com/ to learn more about the services.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Documentation Details

The team has provided the following doc for audit:

1. https://docs.vanarchain.com/documentation-v2/getting-started/vanar

Audit Goals

The focus of the audit was to verify that the system is secure, resilient, and working according to its specifications. The audit activities can be grouped into the following three categories:

1. Security: Identifying security-related issues within the code and the system.
2. Sound Architecture: Evaluation of the architecture of this system through the lens of established best practices and general software best practices.
3. Code Correctness and Quality: A full review of the contract source code. The primary areas of focus include
    a. Correctness
    b. Readability
    c. Sections of code with high complexity
    d. Quantity and quality of test coverage

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Audit Process & Methodology

1. Kickoff Phase:

   During the kickoff phase, the audit team collaborates with the organization to define the scope of the audit. This involves:

   - Defining blockchain objectives, functionalities, and anticipated outcomes.
   - Conducting a comprehensive code walkthrough, reconnaissance, and documentation review.
   - Setting up the codebase for compilation, building, and running tests.
   - Executing executive-level discussions regarding audit objectives and strategies.

2. Audit Process (Security Review):

   The security review phase involves various assessments categorized into automated and manual reviews. Key activities include:

   - Automated testing: Utilizing static analyzers, dynamic testing, and property-based testing.
   - Manual reviews: Conducting line-by-line code scrutiny, testing for edge cases with PoCs and unit tests.
   - Fuzzing campaigns: Employing techniques like invariant testing and symbolic execution.
   - Comparison with known bugs to finalize the audit process.

3. Testing Phase:

   The testing phase involves executing fuzzing campaigns throughout the codebase to verify the underlying logic. Activities include:

   - Employing techniques like invariant testing and symbolic execution.
   - Comparing with known bugs to ensure thorough testing.
   - Documenting and analyzing the results of all campaigns.

4. Reporting Phase:

   During the reporting phase, a detailed report is generated, encompassing:
   - Triaged and classified severity sets for reported findings.
   - Issues identified during testing campaigns, such as fuzzing and symbolic execution.
   - Sharing the initial audit report with the client, marking findings as closed, fixed, or acknowledged based on client responses.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

5. Follow-up Phase:

   The follow-up phase involves:

   - Checking the client's development team's implemented fixes and changes.
   - Ensuring no new bugs are introduced.
   - Releasing the final audit report, providing a comprehensive overview of the blockchain's security posture.

Vulnerability Lists:

In evaluating module-specific vulnerabilities, the following key steps are undertaken:

- Identifying and addressing potential threats across various layers of the blockchain architecture.
- Conducting threat modeling exercises to anticipate attack vectors effectively.

Network Layer Vulnerabilities:

- P2P: Sybil Attack, Eclipse Attack, Eavesdropping Attack, Denial of Service Attack, BGP Hijack Attack, Alien Attack, Timejacking.
- RPC: Eavesdropping Attack, Denial of Service Attack, The Ethereum Black Valentine's Day Vulnerability, HTTP Input Attack, Cross-Domain Phishing Attack.

Ledger Layer Vulnerabilities:

- Consensus: Long Range Attack, Bribery Attack, Race Attack, Liveness Denial, Censorship, Finney Attack, Vector76 Attack, Alternative Historical Attack, 51% Attack, Grinding Attack, Coin Age Accumulation Attack, Selfing Mining, Block Double Production.
- Encryption: Cryptographic Attack, Private Key Prediction, Length Extension Attack, Hash Collision Attack.

Transaction Layer Vulnerabilities:

- Transaction: Transaction Replay Attack, Transaction Malleability Attack, Time-Locked Transaction Attack, False Top-Up Attack, Rug Pull Attack.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Project Summary

Overview

VanarChain is a decentralized blockchain protocol that represents a fork of the renowned Geth client. At the intersection of blockchain, web3, and decentralized technologies, VanarChain strives to provide a borderless and innovative environment for users and developers. The protocol integrates a unique fixed transaction fee mechanism inspired by other prominent blockchain networks like TRON, aiming to enhance accessibility and fairness in transaction pricing.

Technical Features

- **Fixed Transaction Fee Mechanism**:
  VanarChain introduces a fixed transaction fee model, fostering equal pricing for users interacting with the blockchain.

- **Rewards Module**: The protocol includes a rewards module, intending to introduce custom functionality for reward distribution.

- **Transaction Ordering Algorithm**: VanarChain modifies the transaction ordering algorithm, shifting from a dynamic approach to a static FIFO mechanism based solely on transaction time.

Conclusion

VanarChain's ambitious integration of a fixed transaction fee mechanism and other unique features positions it as a player in the evolving landscape of decentralized technologies. To realize its full potential, the protocol must address the identified security, economic, and incentivization challenges, fostering a secure, user-friendly, and competitive blockchain ecosystem.

Note:
The audit scope covers a change log and the specific components of the GETH client implementation modified under the Vanarchain specifications shared privately during the audit cycle.

Additionally, issue responses #1 and #2 mentioned in the subsequent sections of the report require a thorough security review of the codebase after the modifications as conveyed by the developer team, before moving to the production phase.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

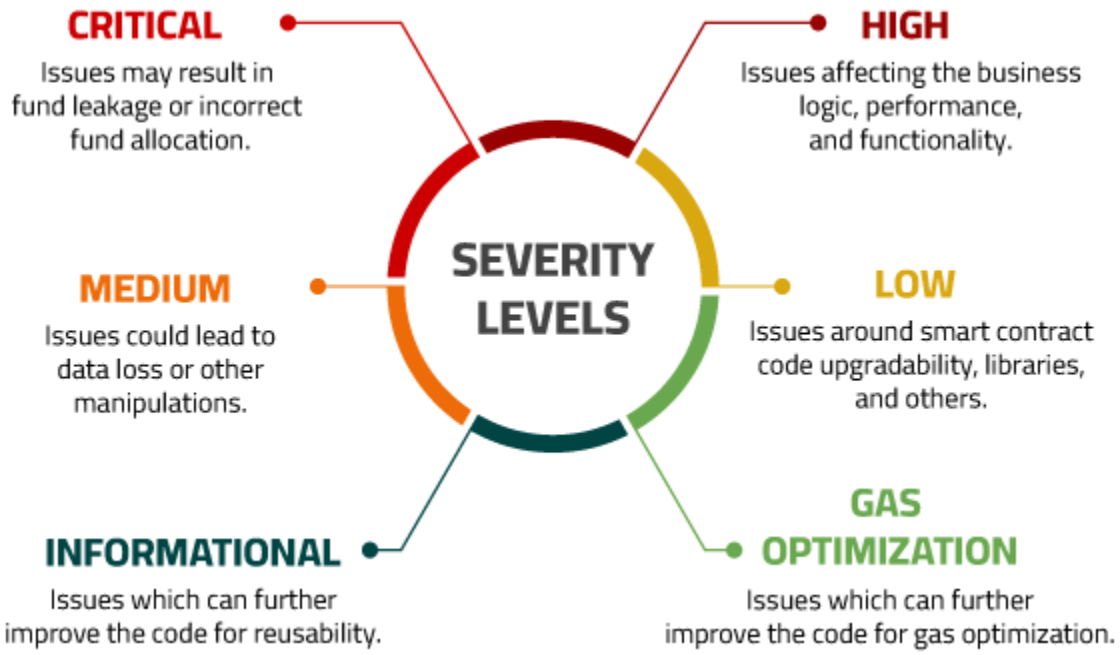
Audit Details

Project Name	Vanarchain
Type	Blockchain
Language	Go
GitHub Link	https://github.com/VanarChain/vanarchain-blockchain/tree/vanarchain-vanguard
Commit	fa64a6851f12bc79b1cccf3f503c274d7bad369c

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Security Level References

Every issue in this report was assigned a severity level from the following:



This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Severity Status References

Severity Status

Issues	Critical	High	Medium	Low	Informational	Gas
Open	-	-	-	-	-	-
Fixed	-	-	-	1	-	-
Redacted	-	-	-	-	-	-
Acknowledged	-	1	1	-	1	-

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Finding

#	Finding	Risk	Status
1	Inadequate Fixed Transaction Fee Mechanism	**High**	**Acknowledged**
2	Improper Miner Incentivization Mechanism and User Motive For Optimized Transactions	**Medium**	**Acknowledged**
3	Incomplete functionality and unused variables	**Low**	**Closed**
4	Unchecked File Overwriting Risk Due to Usage of os.Create()	**Informational**	**Acknowledged**

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Critical Severity Issues

No issues were found.

High Severity Issues

Issue	Inadequate Fixed Transaction Fee Mechanism
Description	The incorporation of a fixed transaction fee mechanism in the Vanar blockchain, a derivative of the geth client, raises concerns due to the lack of comprehensive documentation explaining the underlying motive. Unlike other successful blockchains that meticulously document major upgrades, Vanar's approach lacks transparency, leaving users and third-party auditors in the dark about the rationale behind the fixed fee mechanism. The absence of a clear motive introduces uncertainties and renders the blockchain susceptible to potential misuse by malicious actors.
Impact	The absence of proper documentation significantly undermines the trustworthiness of the Vanar blockchain, particularly concerning the introduction of the fixed transaction fee mechanism. Users, developers, and auditors are left without a clear understanding of the purpose and implications of this significant upgrade. This ambiguity not only discourages new user onboarding but also opens up the protocol to potential misuse by ill-intentioned actors. The impact extends beyond user trust, affecting the overall integrity and security of the blockchain.

Potential Threat Vectors	• **Security Impacts**
	  ○ **DoS Attack Vulnerability:** The fixed fee model exposes the network to Denial of Service (DoS) attacks, allowing malicious actors to flood the system with low-cost, resource-intensive transactions, potentially overwhelming the network without incurring significant costs.
	  ○ **Reduced Incentive for Security Optimization:** In contrast to environments with dynamic fees that encourage users to optimize transactions for efficiency, the fixed fee model diminishes this incentive. This may lead to the deployment of less secure and inefficient smart contracts.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

	• **Economic Impacts**
	
	  ○ **Miner Revenue and Network Health:** Fixed fees may not accurately reflect the actual computational and operational costs, jeopardizing miner revenue and compromising the overall health and security of the network.
	  ○ **User Experience and Transaction Costs:** Users may face uncertainty about the fairness of transaction costs, particularly if the fixed fee does not align with the computational effort required for their transactions.
	
	• **Gaming and Adversarial Impacts**
	
	  ○ **Exploitation by Malicious Actors:** The fixed fee structure could be exploited by attackers to conduct spam or griefing attacks, inundating the network with transactions to disrupt normal operations without incurring proportionate costs.
	  ○ **Manipulation of Transaction Throughput:** In a fixed fee system, certain users or groups could dominate transaction throughput, potentially leading to unequal access to the network and disadvantaged smaller users.

Recommendation	To address these critical concerns, it is imperative that the development team produces a comprehensive and technically detailed documentation package. This documentation should elucidate the motive behind the fixed transaction fee mechanism, providing a clear understanding of its purpose and the considerations that led to its implementation. In addition, a thorough explanation of how this upgrade aligns with the blockchain's objectives and user expectations is crucial. This documentation will not only enhance transparency but will also empower users, developers, and auditors to make informed assessments, fostering trust and mitigating potential security risks associated with the fixed fee model.
Status	**Acknowledged**
Client Response	The rationale and the business logic behind the fixed fees for any transaction in the VANAR ecosystem are well explained in the official documentation. It explains the motive, strength, and key advantages of this feature.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

	At the same time, we also know that the fixed fee at a very low cost welcomes a lot of bad actors to attack the network which needs to be addressed before it can go live (ideally public testnet too should be protected before mainnet). Below are some of the challenges to be addressed and we are working on it to protect the network from any bad actors and possible attacks: - DoS attack - Distributed DoS attack using multiple accounts and different transactions - Humungous-sized transactions consuming full block - It can bring the TPS to 1 transaction per 3 seconds (3 sec is the block time) - Fake transactions to congest the network - pattern recognition We are reviewing a tiered fee structure to help mitigate these possible attack vectors. Also, we are now looking at creating a way to charge more for transactions that are out of bounds to avoid DoS attacks.
Auditor Response	The audit team acknowledges the developer's feedback on the fixed-fee issue as stated above. Additionally, we disclaim that the codebase changes to incorporate the fixed fee mechanism require a thorough review from all of the economic, security, and game-theory perspectives before the blockchain goes into production mode.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Medium severity issues

Issue	**Improper Miner Incentivization Mechanism and User Motive For Optimized Transactions**
Description	The Vanar chain, a fork of the geth client, has documented a modification in the transaction ordering algorithm within its changelog. This code update transitions the transaction ordering mechanism from a dynamic approach, influenced by multiple factors, to a rigid First-In-First-Out (FIFO) mechanism solely based on transaction time. Unlike the geth client, which allows transaction ordering based on user-paid gas fees, the Vanar chain's fork adopts a static approach due to introducing a fixed fee mechanism. As highlighted in the Economic Impacts under the Potential Threat Vectors heading, the protocol fails to establish a proper incentivization mechanism for miners participating in network operations. This deficiency negates the competitive nature essential for mining and validating in a Proof of Authority (PoA) blockchain.
Impact	The implementation of such an algorithm poses risks to the blockchain's adaptability within the broader ecosystem of blockchains. In a landscape where incentives and rational behavior are highly encouraged, a lack of proper incentivization mechanisms can diminish the competitive spirit crucial for the successful operation of a PoA blockchain.
Recommendation	To address this issue, it is strongly recommended that the Vanar chain development team actively works on devising a robust incentivization mechanism. This mechanism should be designed to rekindle the competitive nature of mining and validating in a PoA blockchain, ensuring that miners are motivated to participate in network operations with diligence, ultimately contributing to the network's health, security, and overall success. This proactive approach aligns with industry best practices and fosters a thriving blockchain ecosystem. 1. **<u>TRON's Approach Adaptation</u>** <u>Concept:</u> Adapt TRON's bandwidth and energy system, where accounts are allocated a certain amount of free transaction capacity, with excess usage consuming network tokens.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Technical Justification:

- Resource Allocation: This system ensures fair resource allocation, where each account gets a baseline capacity for transactions, preventing spam by limiting excess usage.
- Token Utilization: Requiring the consumption of native tokens for excess transactions creates a direct cost for spamming the network.

Implementation Considerations:

- Resource Allocation Model: Define how resources (bandwidth and energy in TRON's case) are allocated and consumed.
- Token Economics: Carefully design the token economics to ensure that the cost of excess transactions is significant enough to deter spam but not so high as to prevent legitimate usage.

2. **Stake-Based Transaction Limiting**

Concept:
Implement a mechanism where the ability to send transactions is linked to the stake a user holds in the network. This resembles a proof-of-stake (PoS) model where transaction rights are proportional to the stake.

Technical Justification:

- **Economic Disincentive for Attackers:** Holding a significant stake to spam the network becomes economically unfeasible. Attackers would need to invest heavily in the network token, which is counterintuitive as their actions would devalue their investment.
- **Sybil Attack Mitigation:** This approach inherently mitigates Sybil attacks, where an attacker creates many pseudonymous identities. Since the transaction limit is tied to a stake, creating numerous low-stake identities doesn't increase the attacker's ability to spam the network.

Implementation Considerations:

- **Stake Quantification:** Define how stake is quantified – it could be based on the amount of native token held, duration of holding, or a combination of factors.
- **Dynamic Limits:** Implement dynamic transaction limits based on network conditions. During high congestion, reduce the limit to prioritize genuine transactions.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

3. **Adaptive Fee Adjustment**

Concept:
Introduce a mechanism where the fixed transaction fee can adapt based on real-time network conditions like congestion, average transaction size, and time of day.

Technical Justification:
- **Congestion Pricing:** Similar to real-world congestion pricing, fees can increase during peak times, discouraging spam transactions.
- **Dynamic Network Management:** This system allows the network to respond to changing conditions, maintaining throughput and efficiency.

Implementation Considerations:
- **Algorithm Design:** Develop an algorithm considering various network metrics to adjust fees. This algorithm needs to be transparent and predictable to maintain user trust.
- **Feedback Loop:** Ensure there's a feedback loop where the impact of fee adjustments is monitored and used to refine the algorithm.

4. **Community Governance for Fee Structure**

Concept:
Implement a decentralized governance system where stakeholders can propose and vote on fee structure changes.

Technical Justification:
- **Decentralized Decision-Making:** This approach leverages the community's collective wisdom, leading to more balanced and fair decisions.
- **Responsiveness to Ecosystem Needs:** Community governance allows for quicker adaptation to changing network needs and user preferences.

Implementation Considerations:
- **Voting Mechanism:** Design a robust, tamper-resistant voting system. Consider weighted voting based on stake or contribution to the network.
- **Proposal Thresholds:** Set submission thresholds to prevent spam and ensure only serious suggestions are considered.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

	5. **Enhanced Transaction Monitoring** Concept: Deploy advanced monitoring tools to analyze transaction patterns and flag potential DoS attacks. Technical Justification: • **Anomaly Detection:** By continuously monitoring transaction patterns, the system can quickly identify anomalies indicative of a DoS attack, such as sudden spikes in transaction volume from specific accounts or IPs. • **Preventive Action:** Once a potential attack is detected, the network can take preemptive actions, like temporarily increasing fees for suspicious accounts or IPs or throttling their transaction throughput. Implementation Considerations: • **Data Analysis Tools:** Utilize big data analysis tools and machine learning algorithms to identify abnormal patterns accurately. • **Privacy Preservation:** Ensure monitoring efforts respect user privacy and comply with relevant data protection regulations.
Status	**Acknowledged**
Client Response	Improper Miner Incentivization: Stable Block Rewards: Our 20-year release schedule for block rewards ensures a predictable and stable income for validators. This long-term approach is designed to maintain consistent incentivization, independent of short-term market fluctuations. Transaction Fees and Operational Costs: While transaction fees are low, we ensure they are calibrated to cover the operational costs of validators, safeguarding their financial stability. Market Volatility Mitigation: We recognize the dependency of validator incentives on our native gas token's market price. To mitigate risks associated with market volatility, we are exploring options like partial stabilization mechanisms or providing additional rewards during market downturns. Transition from POA to Decentralized Model: Initially, VANAR operates on a Proof of Authority model managed by the VANAR Foundation. We have a strategic plan for gradual

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

	decentralization, ensuring that as we transition, the incentivization mechanism remains robust and attractive for new validators. The process for this transition will be transparent and involve community input to maintain trust and stability. Developers Motive for Optimized Transactions: Encouraging Quality Smart Contract Development: While we trust developers, we will implement measures to promote optimized transaction code. This includes providing comprehensive guidelines for smart contract development, offering auditing services, and setting up incentive mechanisms for well-optimized contracts. Empowering Users: Responsibility is shared with developers, but we provide them with tools and resources to optimize their transactions. These resources include cost estimators, best practice guides, and developer forums to encourage community collaboration and knowledge sharing. Reviewing multiple levels of fees: Upon reflection we are now considering introducing multiple levels of fees based on transaction type, gas levels and additional indicators before we launch our mainnet. This will help with incentivsation for miners.
Auditor Response	Acknowledged with no further refute.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Low severity issues

Issue	Incomplete functionality and unused variables
Description	Within the changelog of the Vanar chain, a fork of the geth client, the Rewards module attempts to introduce custom functionality for reward distribution. However, during the audit of the code, a couple of deficiencies have been identified. Specifically, the variable "HalvingBlockInterval" is initialized in the Var function but remains unused in the finalize() method. Additionally, the reward distribution algorithm remains unchanged and is labeled as "To Be Determined" (TBD), indicating a pending issue. The initialization of HalvingBlockInterval, represented as HalvingBlockInterval = new(big.Int).SetUint64(1000), does not find practical application in the finalize() method.
Impact	Incomplete functionalities within the blockchain, particularly in critical components like reward distribution, can have adverse effects on user onboarding. Users discovering that the core mechanisms, such as reward distribution, lack feasibility may reconsider their initial interest in becoming part of the blockchain network.
Recommendation	It is strongly recommended that the development team thoroughly document and implement a robust mechanism for reward distribution within the Vanar chain. This documentation should clarify the purpose and functionality of the reward distribution and ensure that new users, during their onboarding process, are presented with an enticing incentive to join the blockchain network. This proactive approach will enhance the user experience and contribute to the sustained growth and attractiveness of the Vanar chain.
Status	Closed
Client Response	This is related to some work-in-progress implementation that is supposed to be completed and ready for the audit in the upcoming audit iteration. Also, the unused variables are removed and they were there because the underlying change was not fully done.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Auditor Response	The audit team considers this issue as closed and marked as out-of-scope from the ongoing security review.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Informational

Issue	Unchecked File Overwriting Risk Due to Usage of os.Create()
Description	The VanarChain codebase exhibits a potential file overwriting vulnerability due to the usage of os.Create() without prior checks for the existence of files. This practice can lead to unintentional overwriting of existing files, posing a risk to data integrity and stability of the blockchain system.

Code Affected	Function: Transition
	traceFile, err := os.Create(path.Join(baseDir, fmt.Sprintf("trace-%d-%v.jsonl", txIndex, txHash.String())))
	if err != nil {
	  return nil, NewError(ErrorIO, fmt.Errorf("failed creating trace-file: %v", err))
	}
	e
	Function: WriteArchive
	archfd, err := os.Create(name)
	if err != nil {
	  return err
	}
	Function: PGPSignFile
	out, err := os.Create(output)
	if err != nil {
	  return err
	}
	
	Function: StartCPUProfile
	f, err := os.Create(expandHome(file))
	if err != nil {
	  return err
	}

Impact	The use of os.Create() without prior existence checks can lead to overwriting of critical files, potentially causing data loss or corruption. In a blockchain context, this could compromise the integrity of the system, lead to inconsistent states, and ultimately undermine trust in the blockchain network.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Recommendation	Replace calls to os.Create() with os.OpenFile() combined with the flags os.O_CREATE|os.O_EXCL. This approach ensures an error is thrown if the file already exists, preventing accidental overwriting. Additionally, consider creating a dedicated file management module to handle file operations securely and consistently throughout the codebase.
Status	**Acknowledged**
Client Response	I appreciate your attention to the potential risks associated with file handling in our system. Specifically, you've pointed out concerns about using os.Create() which could potentially lead to unchecked file overwriting. Let me clarify why this is not an issue in our case: In layman's terms, when our system creates a file, it names each file using a unique identifier - specifically, the transaction hash. Think of a transaction hash like a unique fingerprint for each transaction. Just like no two people have the same fingerprint, no two transactions in our system have the same hash. So, when we use os.Create() to make a new file, the name of the file is this unique transaction hash. Because each transaction hash is unique, each file name is also unique. This means that every time a file is created, it's guaranteed to have a different name from every other file. In a typical scenario where file overwriting is a concern, you might have files with common names like "document1.txt" or "report.pdf", and creating a new file with the same name might accidentally replace the old one. But in our system, since each file is named after a unique transaction hash, it's like naming a file something like "document1-ABC123XYZ.txt" where "ABC123XYZ" is a unique code that never repeats. This unique naming ensures that no new file will ever have the same name as an existing file, thus eliminating the risk of accidentally overwriting an old file with a new one. In summary, the use of unique transaction hashes as file names in our system serves as a safeguard against the risk of file overwriting. Each file created is as unique as a fingerprint, ensuring no overlap or accidental replacement of files. You can see clearly the code passes in the txHash in this case into the file name, so it can't ever overwrite since it is a unique value, used to create each file that is specific to each transaction.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

	Code referenced: traceFile, err := os.Create(path.Join(baseDir, fmt.Sprintf("trace-%d-%v.jsonl", txIndex, txHash.String()))) Finally, this is an operation that we didn't write and is committed to the base geth fork that we are using so it isn't a change we made, nor one that needs to be amended as there is no issue here.
Auditor Response	Closing the issue due to non-relevance.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Concluding Remarks

While conducting the audits of the Vanarchain, it was observed that the system contains High, Medium, and Low severity issues. Our auditors suggest that the developers should resolve issues on priority before moving up with production. The recommendations given will improve the operations of the Blockchain.

Note: The fixes/acknowledgment have been done by the Vanarchain team liaising with ImmuneBytes security engineers.

Disclaimer

ImmuneBytes' audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program complementing this audit is strongly recommended.

Our team does not endorse the Vanarchain platform or its product, nor is this audit investment advice.
Notes:

- Please make sure the code deployed on the mainnet is the ones audited.
- Check for code refactoring by the team on critical issues.

IMMUNEBYTES

AUDITS